САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 3

Контейнеризация написанного приложения средствами Docker

Выполнил:
Люсин Дмитрий
Группа: К3339

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

В рамках лабораторной работы необходимо выполнить контейнеризацию написанного микросервисного приложения средствами Docker. Для каждого сервиса требуется реализовать Dockerfile, написать общий файл docker-compose.yml и настроить сетевое взаимодействие между сервисами.

В качестве основы используется проект приложения бронирования столиков в ресторанах, ранее разделенный на микросервисы и дополненный межсервисным взаимодействием через RabbitMQ.

Ход работы

Для выполнения лабораторной работы была взята версия проекта с микросервисной архитектурой и RabbitMQ. Приложение включает API Gateway, сервисы пользователей, каталога ресторанов, меню, медиа, бронирований, отзывов, уведомлений, а также брокер сообщений RabbitMQ.

Структура контейнеризированного проекта:

```
restaurant-booking-microservices-docker/
├── services/
│   ├── api-gateway/
│   ├── auth-user-service/
│   ├── catalog-service/
│   ├── menu-service/
│   ├── media-service/
│   ├── booking-service/
│   ├── review-service/
│   └── notification-service/
├── docker-compose.yml
├── .env.example
└── README.md
```

Для каждого сервиса был создан отдельный Dockerfile. Такой подход позволяет собирать каждый микросервис как независимый контейнер и запускать его отдельно от остальных частей системы.

Сервис	Dockerfile	Назначение
API Gateway	services/api-gateway/Dockerfile	Единая точка входа для клиента

Auth & User Service	services/auth-user-service/Dockerfile	Регистрация, вход, JWT, пользователи
Catalog Service	services/catalog-service/Dockerfile	Рестораны, кухни, столики
Menu Service	services/menu-service/Dockerfile	Категории меню и блюда
Media Service	services/media-service/Dockerfile	Фотографии ресторанов
Booking Service	services/booking-service/Dockerfile	Бронирования
Review Service	services/review-service/Dockerfile	Отзывы
Notification Service	services/notification-service/Dockerfile	Уведомления по событиям RabbitMQ

Общий запуск приложения описан в файле docker-compose.yml. В нем задаются контейнеры всех микросервисов, RabbitMQ, общая сеть, переменные окружения и volumes для хранения локальных SQLite-баз данных.

Основные элементы docker-compose.yml:

- описаны контейнеры для всех микросервисов приложения;
- добавлен контейнер RabbitMQ с Management UI;
- создана общая сеть restaurant-network для взаимодействия сервисов;
- настроены имена сервисов, по которым контейнеры обращаются друг к другу;
- добавлены volumes для сохранения данных каждого сервиса;
- наружу опубликованы только нужные порты: API Gateway и RabbitMQ Management UI.

Внутри Docker-сети сервисы обращаются друг к другу не через localhost, а по именам контейнеров. Например, Booking Service может обращаться к Auth & User Service по адресу http://auth-user-service:3001, а к RabbitMQ - по адресу amqp://rabbitmq:5672.

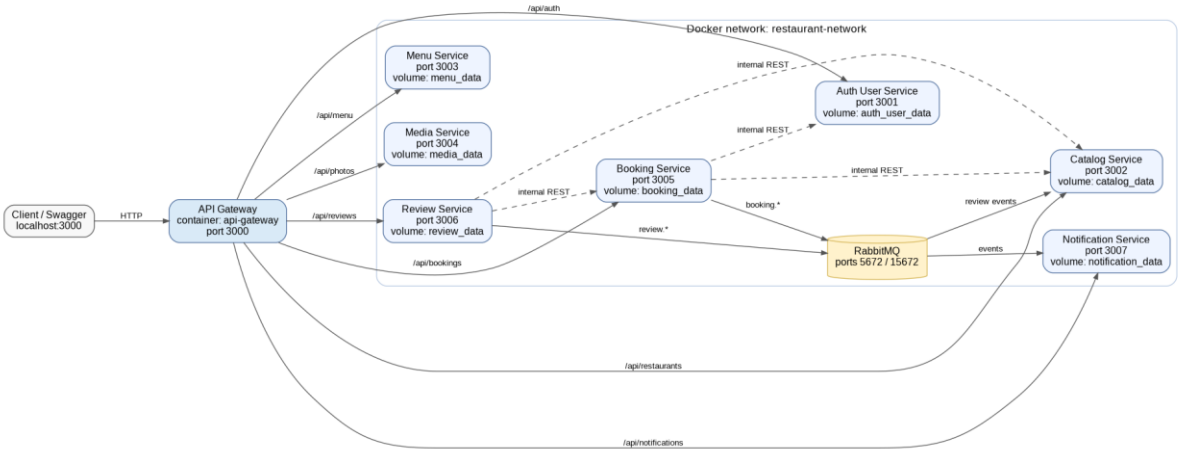


Рисунок 1 - Схема контейнеризации микросервисного приложения

После запуска доступны следующие адреса:

Компонент	**Адрес**
API Gateway	http://localhost:3000
Swagger	http://localhost:3000/api-docs
Health check	http://localhost:3000/health
RabbitMQ Management UI	http://localhost:15672

Вывод

В результате лабораторной работы была выполнена контейнеризация микросервисного приложения средствами Docker. Для каждого сервиса создан отдельный Dockerfile, а общий запуск всех компонентов описан в docker-compose.yml.

Сервисы объединены в общую Docker-сеть и взаимодействуют друг с другом по именам контейнеров. Для хранения данных используются отдельные volumes, что сохраняет принцип database-per-service. Контейнеризация позволяет запускать все приложение одной командой и упрощает проверку проекта на другом компьютере.